

Automated Teller Machine In Java

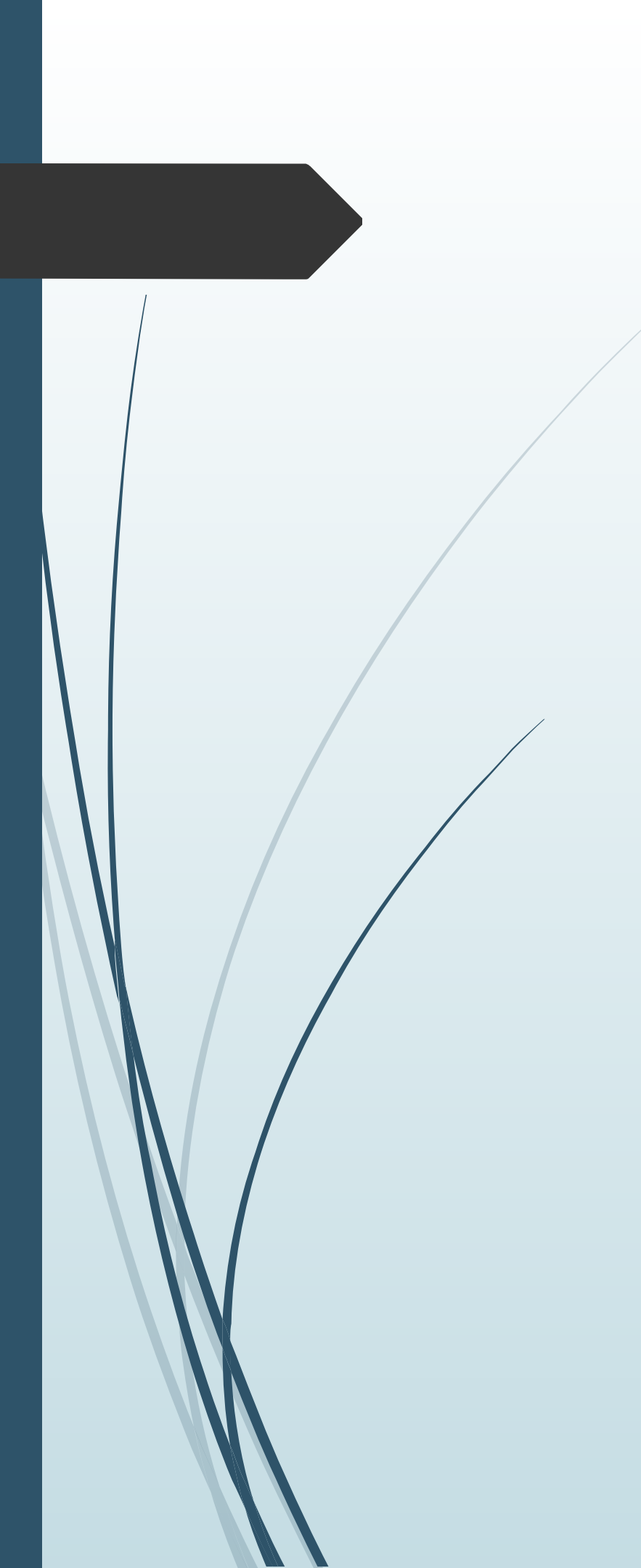
PRESENTED BY:

PEETH CHOWDHARY - 16010122034

DHRUV DESAI - 16010122038

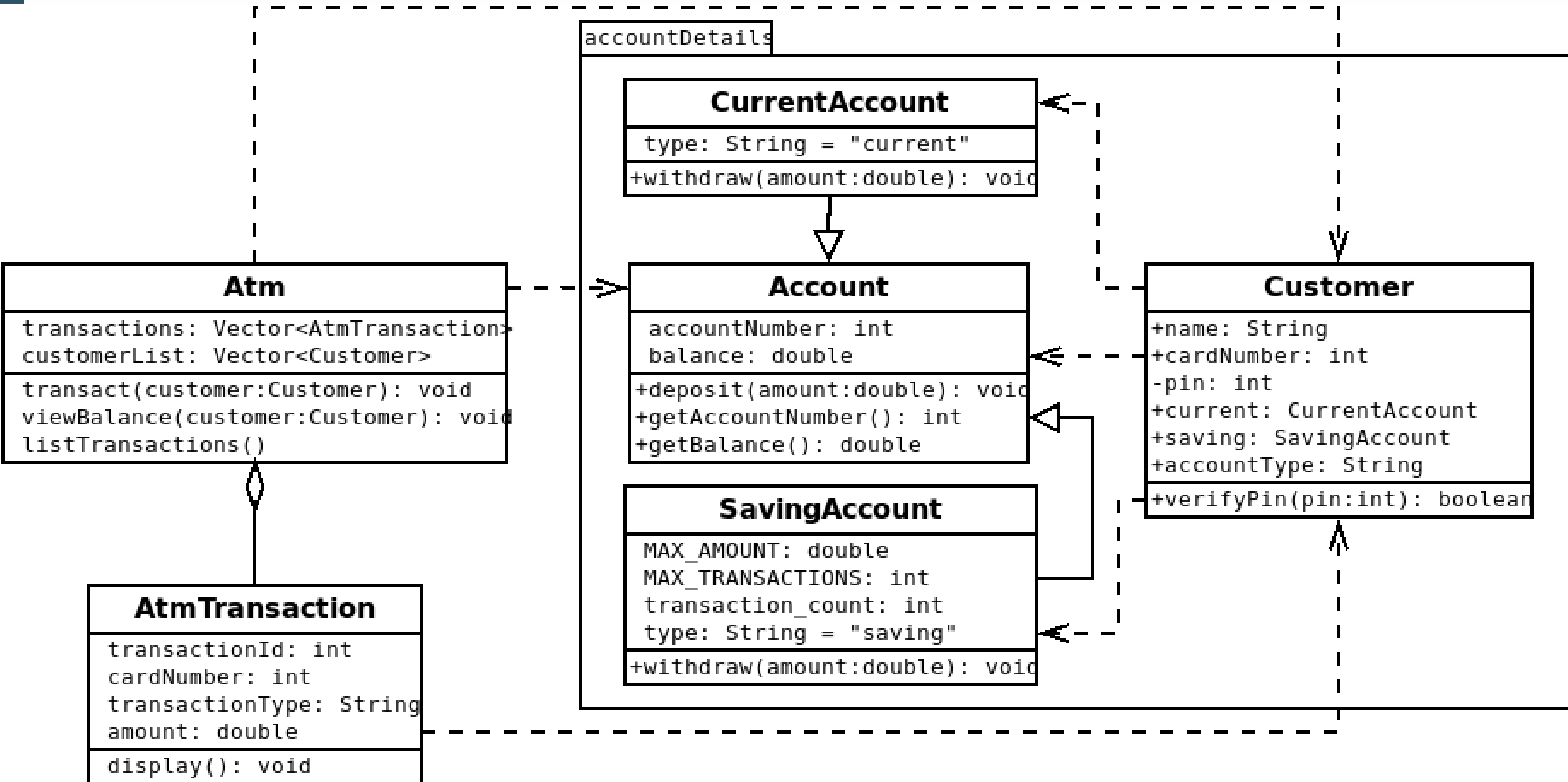
BACKGROUND

- In the modern banking landscape, Automated Teller Machines (ATMs) serve as essential self-service points for customers to conduct various financial transactions. A bank has recently implemented a new ATM system to cater to the needs of its customers. The system allows pre-existing bank customers to perform the following transactions:
 1. **Deposit Money:** Customers can deposit funds into their bank accounts via the ATM.
 2. **Withdraw Money:** Customers can withdraw funds from their bank accounts using the ATM.
 3. **View Transactions:** Customers can view their transaction history to keep track of their financial activities.
- **Account Types:**
 - The bank offers two types of accounts:
 4. **Current Account:** This account type offers unlimited transactions and has no specific transaction limits. It is designed for customers who require easy access to their funds.
 5. **Savings Account:** This account type has a maximum transaction limit. It is intended for customers who want to save money while still having some transaction flexibility.



PROBLEM STATEMENT

- The goal of this project is to design and implement a class diagram for the ATM system that allows customers to interact with their bank accounts through the ATM.
- The system should provide a user-friendly interface for customers to deposit, withdraw, and view transactions.
- It should also enforce the account-specific transaction limits for savings accounts.





FEATURES

Inheritance: SavingAccount and CurrentAccount classes inherit from Account class.

Dependencies: The Atm and AtmTransaction classes are dependent on the Customer class while the Customer class is dependent on the Account, SavingAccount and CurrentAccount classes.

Aggregation: There is an aggregation relationship between Atm class and AtmTransaction class

Packages: The only user defined package in the clas diagram is accountDetails. It holds Customer, Account, SavingAccount and CurrentAccount classes.



IMPLEMENTATION DETAILS

```
package accountDetails;
```

```
public class Account{
```

```
    int accountNumber;
```

```
    double balance;
```

```
    Account(int accNum, double balance){
```

```
        accountNumber = accNum;
```

```
        this.balance = balance;
```

```
    }
```

```
    public void deposit(double amount){
```

```
        balance += amount;
```

```
        System.out.println(amount + " Deposited to account " + accountNumber);
```

```
    }
```

```
    public int getAccountNumber(){
```

```
        return accountNumber;
```

```
    }
```

```
    public double getBalance(){
```

```
        return balance;
```

```
    }
```

```
}
```

```
package accountDetails;
```

```
public class CurrentAccount extends Account{
```

```
    String type = "current";
```

```
    CurrentAccount(int accNum, double balance){
```

```
        super(accNum, balance);
```

```
    }
```

```
    public void withdraw(double amount){
```

```
        balance -= amount;
```

```
        System.out.println(amount + " Withdrawn from account " + accountNumber);
```

```
    }
```

```
}
```

```
package accountDetails;

public class SavingAccount extends Account{
    final double MAX_AMOUNT;
    final int MAX_TRANSACTIONS;
    int transactionCount;
    String type = "saving";

    SavingAccount(int accNum, double balance, double maxAmount, int maxTransactions){
        super(accNum, balance);
        MAX_AMOUNT = maxAmount;
        MAX_TRANSACTIONS = maxTransactions;
    }

    public void withdraw(double amount){
        if(transactionCount >= MAX_TRANSACTIONS){
            System.out.println("Reached maximum number of daily withdrawals. Can not withdraw any more for the day.");
            return;
        }

        if(amount > MAX_AMOUNT){
            System.out.println("Can not withdraw " + amount + ".Withdrawal Limit for account is set to " + MAX_AMOUNT);
            return;
        }

        balance -= amount;
        transactionCount += 1;
        System.out.println(amount + " Withdrawn from account " + accountNumber);
    }
}
```



```
package accountDetails;

public class Customer{
    public String name;
    public int cardNumber;
    private int pin;
    public CurrentAccount current;
    public SavingAccount saving;
    public String accountType;

    public Customer(String name, String accountType, int cardNumber, int pin, double balance){
        this.name = name;
        this.cardNumber = cardNumber;
        this.pin = pin;
        this.accountType = accountType;

        if(accountType.equals("current")){
            current = new CurrentAccount(cardNumber, balance);
            saving = null;
        }
        else{
            saving = new SavingAccount(cardNumber, balance, 5000, 3);
            current = null;
        }
    }

    public boolean verifyPin(int pin){
        return this.pin == pin;
    }
}
```

```
import accountDetails.*;

import java.util.Scanner;
import java.util.Vector;

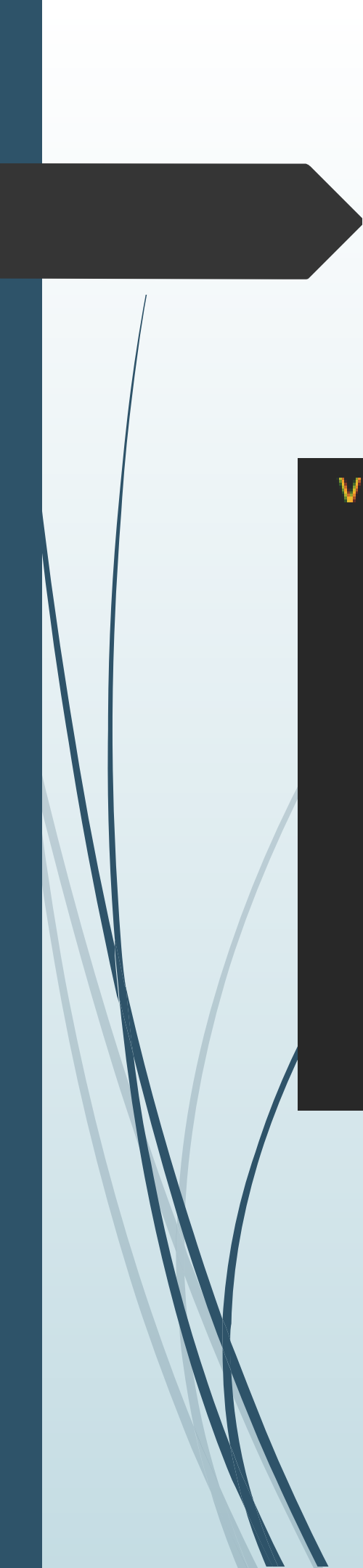
public class Atm{
    Vector<AtmTransaction> transactions = new Vector<AtmTransaction>(5,5);
    Vector<Customer> customerList = new Vector<Customer>(3);

    Atm(){
        customerList.add(new Customer("Peeth", "current", 1234, 1234, 25000.00));
        customerList.add(new Customer("Dhruv", "saving", 7890, 4567, 30000.00));
        customerList.add(new Customer("admin", "saving", 0000, 0000, 0));
    }

    class AtmTransaction{
        int transactionId;
        int cardNumber;
        String transactionType;
        double amount;

        AtmTransaction(int id, int cardNum, String type, double amount){
            transactionId = id;
            cardNumber = cardNum;
            transactionType = type;
            this.amount = amount;
        }

        void display(){
            System.out.println("Transaction ID: " + transactionId);
            System.out.println("Card Associated with Transaction: " + cardNumber);
            System.out.println("Type of Transaction: " + transactionType);
            System.out.println("Amount transacted: " + amount);
        }
    }
}
```



```
void transact(Customer customer, Scanner sc){
    int option;
    while(true){
        System.out.println("Enter Number to do the Associated Transaction:");
        System.out.println("[1] Withdraw");
        System.out.println("[2] Deposit");
        option = sc.nextInt();
        if(option == 1 || option == 2){
            break;
        }
        System.out.println("Enter a valid option.");
    }
}
```

```
double amount;
String transactionType;
if(option == 1){
    while(true){
        System.out.println("Enter Amount to Withdraw: ");
        amount = sc.nextDouble();
        if(amount < 0){
            System.out.println("Amount can not be less than zero.");
            continue;
        }
        break;
    }
    if(customer.accountType.equals("current"))
        customer.current.withdraw(amount);
    else
        customer.saving.withdraw(amount);
    transactionType = "Withdrawal";
}
else{
    while(true){
        System.out.println("Enter Amount to Withdraw: ");
        amount = sc.nextDouble();
        if(amount < 0){
            System.out.println("Amount can not be less than zero.");
            continue;
        }
        break;
    }
    if(customer.accountType.equals("current"))
        customer.current.deposit(amount);
    else
        customer.saving.deposit(amount);
    transactionType = "Deposit";
}
transactions.add(new AtmTransaction(transactions.size() + 1, customer.cardNumber, transactionType, amount));
}
```

```
void viewBalance(Customer customer){
    int accountNum;
    String balance;
    if(customer.accountType.equals("current")){
        accountNum = customer.current.getAccountNumber();
        balance = String.format("%.02f", customer.current.getBalance());
    }
    else{
        accountNum = customer.saving.getAccountNumber();
        balance = String.format("%.02f", customer.saving.getBalance());
    }
    System.out.println("Balance in account " + accountNum + ": " + balance);
}

void listTransactions(){
    for(AtmTransaction transaction : transactions){
        System.out.println();
        transaction.display();
        System.out.println();
    }
}
```

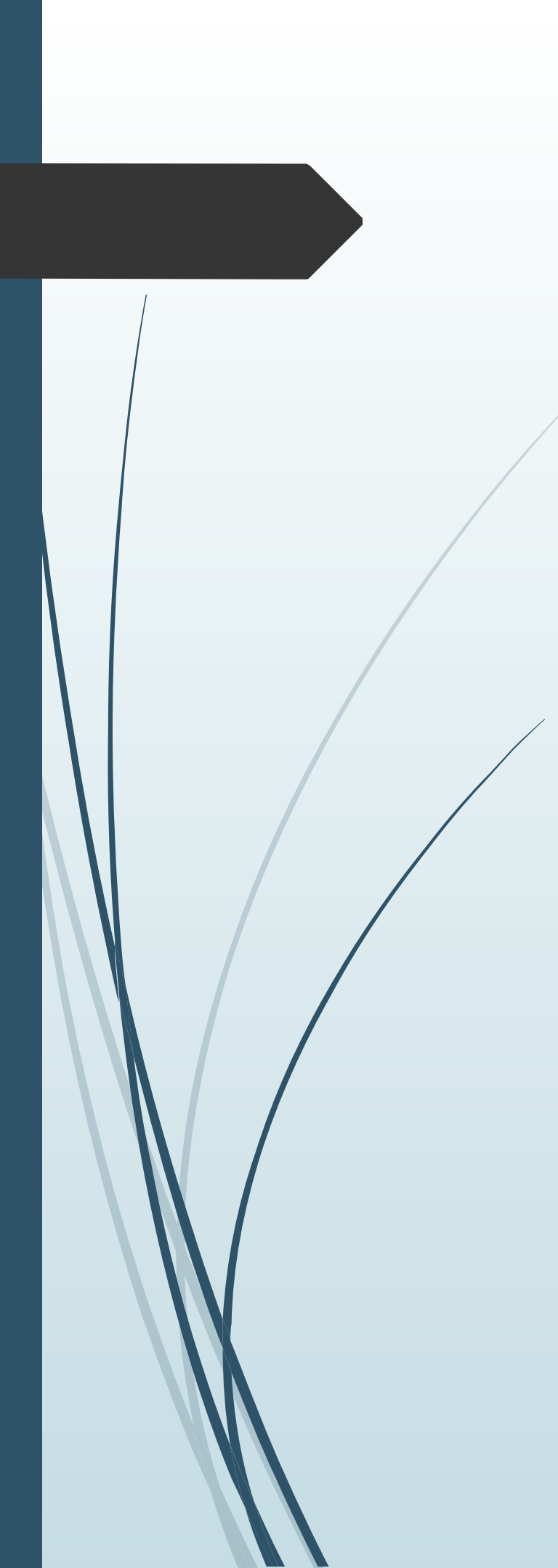


```
public static void main(String[] args){
    Scanner sc = new Scanner(System.in);
    Atm atm = new Atm();
    while(true){
        System.out.println("\n-----ATM-----");
        System.out.println("Enter Card Number: ");
        int cardNum = sc.nextInt();

        int customerIndex = 0;

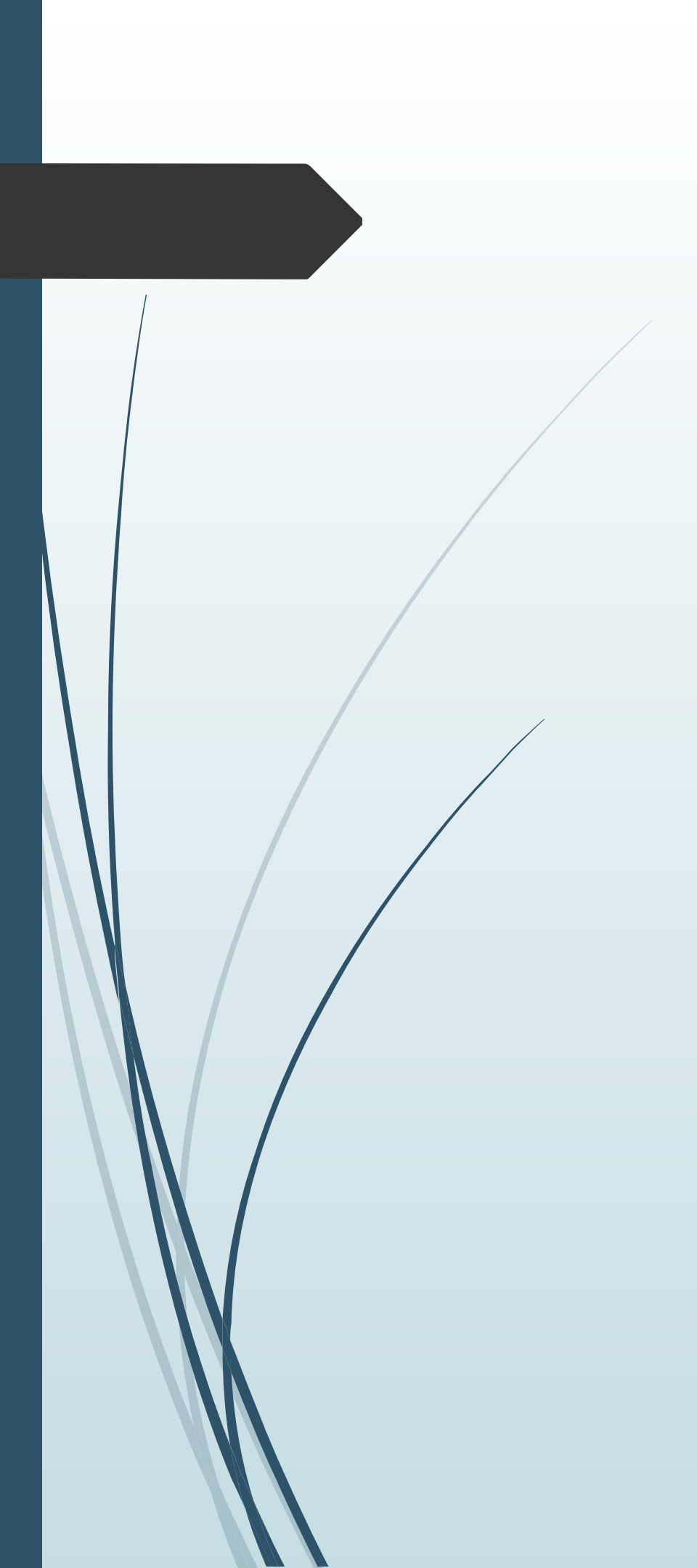
        for(int i = 0; i < atm.customerList.size(); i++){
            if(cardNum == atm.customerList.get(i).cardNumber){
                customerIndex = i;
                break;
            }
        }

        System.out.println("Enter PIN: ");
        int pin = sc.nextInt();
    }
}
```



```
if(atm.customerList.get(customerIndex).verifyPin(pin)){
    if(atm.customerList.get(customerIndex).name.equals("admin")){
        atm.listTransactions();
        System.out.println("Shutdown ATM [y/N]?");
        String shutdown = sc.next();
        if(shutdown.equals("y")){
            System.exit(0);
        }
    }
    else{
        boolean check = true;
        while(check){
            System.out.println("Enter Number to do the Associated Action:");
            System.out.println("[1] Transact (Withdraw/Deposit)");
            System.out.println("[2] View Balance");
            String option = sc.next();
            switch(option){
                case "1":
                    atm.transact(atm.customerList.get(customerIndex), sc);
                    check = false;
                    break;
                case "2":
                    atm.viewBalance(atm.customerList.get(customerIndex));
                    check = false;
                    break;
                default:
                    System.out.println("Enter a valid option.");
            }
        }
    }
}
else{
    System.out.println("Incorrect PIN. Exiting Session.");
}

System.out.println("-----\n");
}
```



TEST CASES


```

-----ATM-----
Enter Card Number:1234
12341      MAX_AMOUNT = maxAmount;
Enter PIN:  4567
12343      MAX_TRANSACTIONS = maxTransactions;
Enter Number to do the Associated Action:
[1] Transact (Withdraw/Deposit)
[2] View Balance
1 17      System.out.println("Reached maximum number of daily withdrawals");
Enter Number to do the Associated Transaction:
[1] Withdraw
[2] Deposit
1 20      return;
Enter Amount to Withdraw:
7000
7000.0 Withdrawn from account 1234

```

Test 1

```

-----ATM-----
Enter Card Number:3456
3456      public String name;
Enter PIN:  345
345      public int cardNumber;
Incorrect PIN. Exiting Session.
-----ATM-----
1 package accountDetails;
2
3 public class Customer{
4     public String name;
5     public int cardNumber;
6     private int pin;
7     public CurrentAccount current;
8     public SavingAccount saving;
9     public String accountType;
10
11     public Customer(String name, String accountType, int cardNumber, int pin, CurrentAccount current, SavingAccount saving){
12         this.name = name;
13         this.accountType = accountType;
14         this.cardNumber = cardNumber;
15         this.pin = pin;
16         this.current = current;
17         this.saving = saving;
18     }
19 }

```

Test 2

```

-----ATM-----
Enter Card Number:7890
7890      count(int accNum, double balance, double maxAmount, int maxTransactions){
Enter PIN:  4567
4567      super(accNum, balance);
1234      MAX_AMOUNT = maxAmount;
Enter Number to do the Associated Action:
[1] Transact (Withdraw/Deposit)
[2] View Balance
1 15      public void withdraw(double amount){
Enter Number to do the Associated Transaction:
[1] Withdraw
[2] Deposit
1 18      System.out.println("Reached maximum number of daily withdrawals. Can not withdraw any more for the day.");
Enter Amount to Withdraw:
8000
Can not withdraw 8000.0! Withdrawal Limit for account is set to 5000.0!
-----ATM-----

```

Test 3

```

-----ATM-----
Enter Card Number:7890
7890      package accountDetails;
Enter PIN:  4567
4567      public class Customer{
Enter Number to do the Associated Action:
[1] Transact (Withdraw/Deposit)
[2] View Balance
1 8      public int cardNumber;
Enter Number to do the Associated Transaction:
[1] Withdraw
[2] Deposit
1 12      public Customer(String name, String accountType, int cardNumber, int pin, CurrentAccount current, SavingAccount saving){
Enter Amount to Withdraw:
500
500.0 Withdrawn from account 7890
-----ATM-----

```

Test 4

```

----15-----public ATMoid-Withdraw(double amount){
Enter Card Number:transactionCount >= MAX_TRANSACTIONS){
78907         System.out.println("Reached maximum number of daily withdrawals. Can not withdraw any more for the day.");
Enter PIN:         . Can not withdraw any more for the day.");
45678         return;
Enter Number to do the Associated Action:
[1]2Transact (Withdraw/Deposit)
[2]3View Balanceaccount.java
5
Enter a valid option.
Enter Number to do the Associated Action:
[1] Transact (Withdraw/Deposit)
[2] View Balance
1
Enter Number to do the Associated Transaction:
[1] Withdraw
[2] Deposit
500
Enter a valid option.
Enter Number to do the Associated Transaction:
[1] Withdraw
[2] Deposit
1
Enter Amount to Withdraw:
500
500.0 Withdrawn from account 7890
-----

```

Test 5

```

-----int--ATM--actionCount;--
Enter CardNumber:ype = "saving";
78908
Enter PIN:SavingAccount(int accNum, double balance, double maxAmount, int maxTransactions){
4567         actions){
Enter Number to do the Associated Action:
[1]1Transact (Withdraw/Deposit)ount;
[2]2View BalanceX_TRANSACTIONS = maxTransactions;
13         }
1 14
Enter Number to do the Associated Transaction:
[1]1Withdraw if(transactionCount >= MAX_TRANSACTIONS){
[2]2Deposit         System.out.println("Reached maximum number of daily withdrawals
1         . Can not withdraw any more for the day.");
Enter Amount to Withdraw:
50019         }
500.0 Withdrawn from account 7890
-----SavingAccount.java-----

-----ATM-----
Enter Card Number:
78901 package accountDetails;
Enter PIN:
45672 public class Customer{
Enter Number to do the Associated Action:
[1]1Transact (Withdraw/Deposit);
[2]2View Balance
13         private int pin;
Enter Number to do the Associated Transaction:
[1]1Withdraw
[2]2Deposit
14         public CurrentAccount current;
Enter Amount to Withdraw:
50015         public SavingAccount saving;
Reached maximum number of daily withdrawals. Can not withdraw any more for the day.
-----
15         public Customer(String name, String accountType, int cardNumber, int pin, double balance){
this.name = name;
this.cardNumber = cardNumber;
this.pin = pin;
this.accountType = accountType;

```

Test 4 Continued and Desired Result

```

-----ATM-----
Enter Card Number:
1234
Enter PIN:
1234
Enter Number to do the Associated Action:
[1] Transact (Withdraw/Deposit)
[2] View Balance
2
Balance in account 1234: 18000.00

-----ATM-----
Enter Card Number:
7890
Enter PIN:
4567
Enter Number to do the Associated Action:
[1] Transact (Withdraw/Deposit)
[2] View Balance
2
Balance in account 7890: 28500.00

```

Test 6.

Test 7.

```

-----ATM-----
Enter Card Number:
0000
Enter PIN:
0000
Transaction ID:
Card Associated with Transaction:
Type of Transaction:
Amount transacted: 7000.0

Transaction ID:
Card Associated with Transaction:
Type of Transaction:
Amount transacted: 8000.0

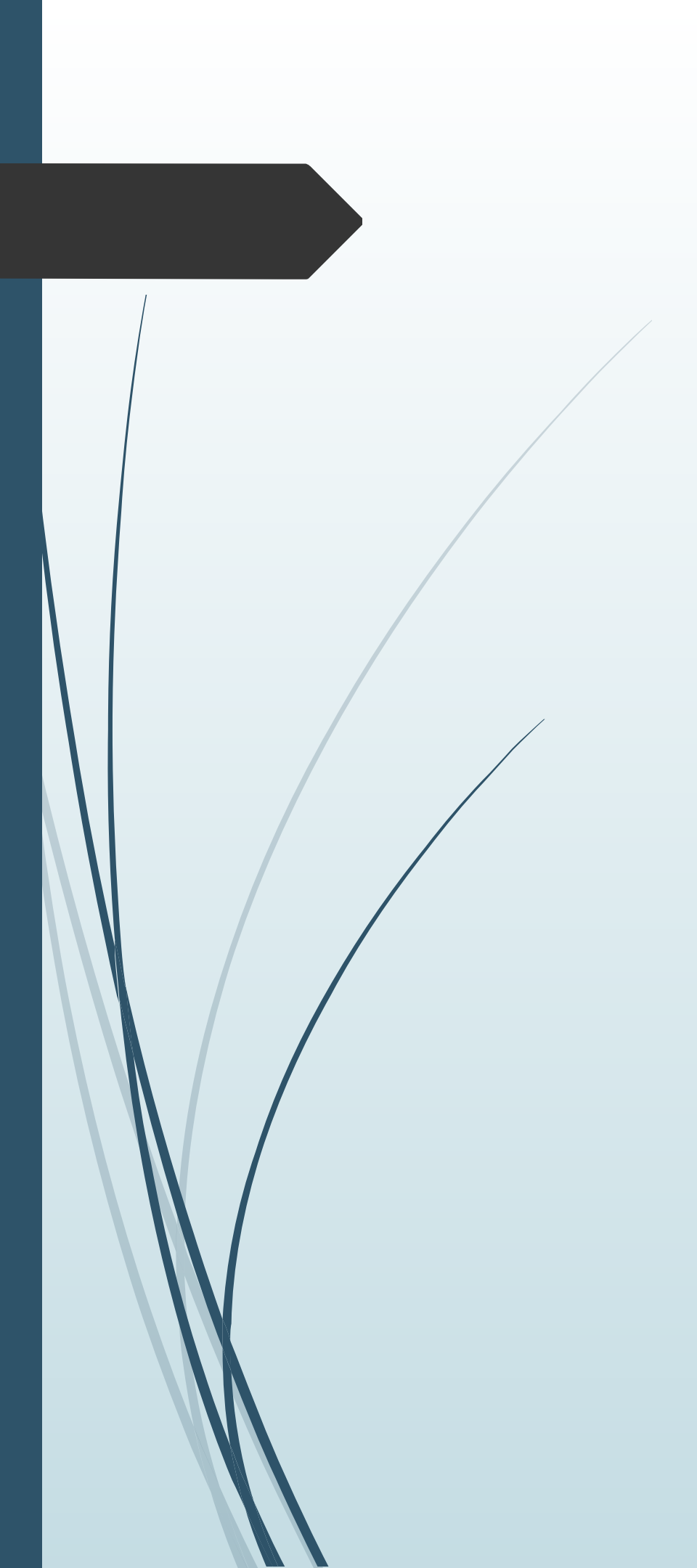
Transaction ID:
Card Associated with Transaction:
Type of Transaction:
Amount transacted: 500.0

Transaction ID:
Card Associated with Transaction:
Type of Transaction:
Amount transacted: 500.0

Transaction ID:
Card Associated with Transaction:
Type of Transaction:
Amount transacted: 500.0

Shutdown ATM [y/N]?

```



```
-----ATM-----
Enter Card Number:
0000
Enter PIN:
0000
Shutdown ATM [y/N]?
y
```

Test 8

```

-----ATM-----
Enter Card Number:
1234
Enter PIN:
1234
Enter Number to do the Associated Action:
[1] Transact (Withdraw/Deposit)
[2] View Balance
2
Balance in account 1234: 25000.00
-----ATM-----
Enter Card Number:
1234
Enter PIN:
1234
Enter Number to do the Associated Action:
[1] Transact (Withdraw/Deposit)
[2] View Balance
1
Enter Number to do the Associated Transaction:
[1] Withdraw
[2] Deposit
2
Enter Amount to Withdraw:
3000
3000.0 Deposited to account 1234
-----ATM-----

```

Test 9

```

-----ATM-----
Enter Card Number:
1234
Enter PIN:
1234
Enter Number to do the Associated Action:
[1] Transact (Withdraw/Deposit)
[2] View Balance
2
Balance in account 1234: 28000.00
-----ATM-----

```


LEARNING OUTCOMES

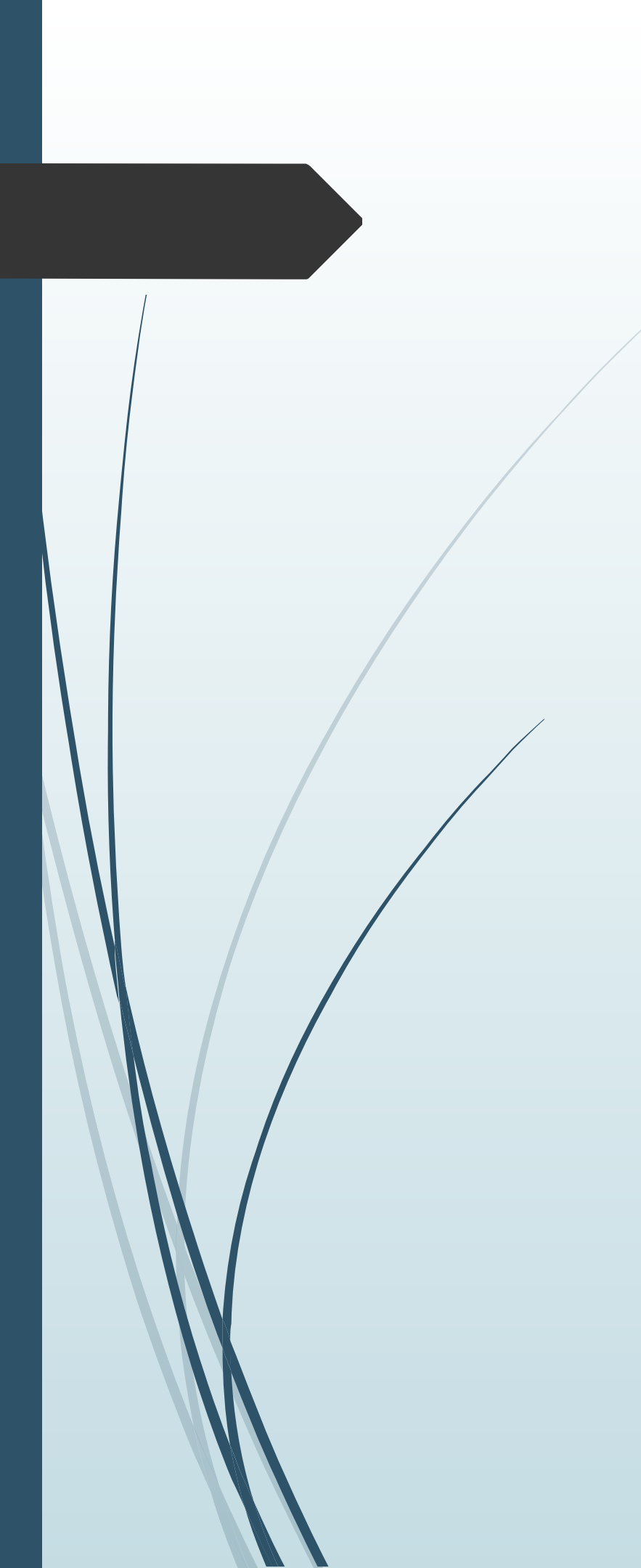
Creating a class diagram for an ATM system helped us better understand key Object-Oriented Programming (OOP) concepts as well as inbuilt functions, keywords and their proper utilization.

Abstraction: Simplifying a system by defining classes that represent real-world entities.

Encapsulation: Bundling data and methods within a class to preserving data integrity specially for the use case of the PINs associated with each ATM card.

Inheritance and Modularity: Creating relationships between classes, allowing one to inherit attributes and methods from another in a way that reduces the need to repeat the code we had written and, at the same time, increase the code's efficiency and readability.

We also learnt how to better use vectors and attributes of objects with the help of a user defined package in classes outside the package we defined.



CONCLUSION

- The successful implementation of the ATM system will provide bank customers with a convenient and secure way to manage their accounts, make deposits, withdraw money, and keep track of their financial transactions. It should ensure that all transactions are accurate and compliant with the account-specific limits, contributing to an enhanced customer experience while maintaining the bank's security and regulatory standards.
- The designed class diagram will serve as the foundation for implementing this ATM system, allowing for the creation of the necessary classes, their relationships, and methods for efficient and secure customer transactions.